

Le Mans Université

Licence Informatique *2ème année*

Module 174UPO2 Rapport de Projet

MAZE-SHOOTER

DEQUIREZ Thomas

JAMMES Corentin

ASTIER Hugues

8 avril 2026

Table des matières

1	Introduction	2
2	Conception	2
2.1	Cahier des charges	2
2.2	Fonctionnalités du jeu	2
3	Organisation du projet	2
3.1	Repartition des tâches	2
3.2	Planning prévisionnel	2
3.3	Outils utilisés	2
4	Développement	2
4.1	Logique d'exécution du jeu et fonctionnement du menu	2
4.2	Création de la carte	3
4.3	Joueur, déplacement et gestion de la vie	3
4.4	Bots, leurs IA, spawn	3
4.5	Tir et armes	3
4.6	Multijoueur, réseaux	3
4.7	Interface graphique, sons	3
5	Résultats et conclusion	3
5.1	Bilan temporel	3
5.2	Bilan de la conformité aux cahiers des charges	3
5.3	Améliorations possibles	3
6	Annexes	3

1 Introduction

2 Conception

2.1 Cahier des charges

Le cahier des charges à respecter suit les points suivants :

- Réaliser un jeu par groupe (de trois ici)
- Utiliser le langage C
- Utiliser une librairie graphique comme SDL ou RAYLIB
- Se servir de GIT
- Suivre un model de gestion de projet
- Utiliser doxygen

On a utilisé la librairie graphique Raylib, ainsi que GIT avec quatre branche (master, corentin, Thomas et HUGues) et on a suivi modèle linéaire.

2.2 Fonctionnalités du jeu

3 Organisation du projet

3.1 Repartition des tâches

3.2 Planning prévisionnel

3.3 Outils utilisés

4 Développement

4.1 Logique d'exécution du jeu et fonctionnement du menu

Dans cette section on explique la logique de fonctionnement du fichier main.c, et par la même occasion, la navigation du joueur dans le menu du jeu. Ce menu permet d'afficher différents boutons utiles à la navigation du joueur CF photo ANNEXE

Pour comprendre comment fonctionnent la logique du jeu dans main.c, on a besoin d'une variable importante : CurrentScreen, c'est un pointeur sur un Gamescreen, qui décrit l'état de jeu actuel. Cette variable peut prendre diverses valeurs comme NOUVELLE PARTIE, CHARGER PARTIE ou encore MENU. Par défaut currentScreen vaut MENU. Il faut maintenant comprendre comment le jeu tourne. Pour cela, dans main.c, on utilise une grande boucle while tournant tant qu'une variable running n'est pas mise à FALSE (ou que le joueur ne supprime pas sa fenêtre). Dans cette boucle il faut distinguer la partie logique du jeu de la partie dessin, chaque partie correspond à un switch case. Chaque switch case regarde que vaut la variable CurrentScreen. Dans la partie logique on met à jour les variables globales, on appel

des fonctions autonomes qui initialisent ou mettent à jours nos variables globales en fonction de différents test. Un bon exemple de fonction destiné à la partie logique est `UpdateGame(...)` (HREF SUR ARGU ET COMM EN ANNEXE) qui mets à jours toute la logique du jeu en mode solo. Une fois la logique à jours un peut dessiner. Le second switch entre en jeu et permet de lancer les bonne fonction de dessin toujours en fonction de la valeurs de `CurrentScreen` Desormais, pour le menu, il suffit de voir que lorsque `CurrentScreen` vaut `MENU` on appel les fonctions gerant la souris, le clavier et on dessinent les boutons voulus.

4.2 Création de la carte

4.3 Joueur, déplacement et gestion de la vie

4.4 Bots, leurs IA, spawn

4.5 Tir et armes

4.6 Multijoueur, réseaux

4.7 Interface graphique, sons

Cette section présente l'implementation des textures des sons , il existe plusieurs manieres d'en créer et de les gérer grace à Raylib.

5 Résultats et conclusion

5.1 Bilan temporel

5.2 Bilan de la conformité aux cahiers des charges

5.3 Améliorations possibles

6 Annexes